

Lab 01 — Bitcoin Core in Regtest

UTXO state and raw transactions

Khusan Ashuraliev · Introduction to Blockchain Programming · Óbuda University, NIK · October 2026

Item	Value
Repository	https://github.com/KhusanAshuraliev/lab-01-bitcoin-regtest
Node	Bitcoin Core /Satoshi:27.1.0/ (image lncm/bitcoind:v27.1), regtest, Docker on macOS
Evidence	Every command and its full output: evidence/transcript.txt; one file per step and evidence/summary.json in the repository
Reproduce	<code>uv run python lab/run_lab.py --lab-dir <OE_BLOCKCHAIN>/Labs/lab-01-bitcoin-regtest</code>
Tests	Own parser: 91 passed (the course suite of 31 tests + 60 own). Reference parser: 31 passed .

All values below are copied from this run's evidence files. JSON arguments were passed to `bitcoin-cli` with `-stdin` to avoid shell quoting; otherwise the commands are exactly those of the lab README.

1. Setup and Part A — the UTXO state after mining

The setup created the descriptor wallet `lab01`, generated a mining address and mined 101 blocks to it (height 101). `getbalance` then reported **50.00000000** BTC.

Output of `listunspent`

```
$ docker compose exec -T bitcoind bitcoin-cli -rpcwallet=lab01 listunspent
[
  {
    "txid": "3b4aa4fcb007fa26cb484cc0164708cb39aaf3f0b98fb8643970e1649fc15b9f",
    "vout": 0,
    "address": "bcrt1ql8q39jdahzr6mr4tzkzvqgk5cdps9f9ltn47xk",
    "label": "",
    "scriptPubKey": "0014f9c112c9bdb887ad8eab1584c402d4c34302a4bf",
    "amount": 50.00000000,
    "confirmations": 101,
    "spendable": true,
    "solvable": true,
    "desc": "wpkh([0890d39b/84h/1h/0h/0/1]026ebd344d011c1de4835bb1c56fd295447e363c551fdb110bca2edab558f63f0f)#s3n7yq7w",
    "parent_descs": [
      "wpkh(tpubD6NzVbkrYhZ4WzfKfEPtmXVyBpNa8Fr2y7j8ux68JnTwyGq4fDirSRKX6gu5po5XVDjn1aCU1U6N6uo4kiKNSaJydWsv9f2ZJx1uiWLZb2m/84h/1h/0h/0/*)#larsdd72"
    ],
    "safe": true
  }
]
```

Command	Result	README expects
<code>listunspent</code>	1 entry — txid 3b4aa4fc...9fc15b9f, vout 0, 50 BTC, 101 confirmations, spendable	1 entry, 50, 101, spendable ✓
<code>getbalances</code>	trusted 50.00000000 , immature 5000.00000000 , untrusted_pending 0	50 / 5000 ✓
<code>gettxoutsetinfo</code>	height 101, txouts 101 , transactions 101, total_amount 5050.00000000	101 / 101 / 5050 ✓

Why 100 of the 101 mined outputs are unspendable

Each of the 101 blocks has a coinbase transaction paying the 50 BTC subsidy to my mining address, so 101 outputs worth 5 050 BTC exist. Consensus forbids spending a coinbase output until it has **100**

confirmations on top of it (COINBASE_MATURITY). At height 101 only the block-1 coinbase satisfies this (it has 101 confirmations); the coinbases of blocks 2–101 have between 100 and 1 confirmations and are *immature*. The rule exists because a chain reorganisation removes the coinbase of an orphaned block: without a maturity delay, coins that later vanish could already have been spent onward, and every descendant transaction would become invalid.

The wallet and the node look at the same data from two angles. `listunspent` lists only outputs the wallet can spend *now*, and never includes immature coinbase outputs, which is why it shows one entry; `getbalances` reports the other $100 \times 50 = 5\,000$ BTC under *immature*. `gettxoutsetinfo` is the node's view of the whole UTXO set, independent of any wallet: it counts every unspent output regardless of owner or maturity, hence **101** outputs and 5 050 BTC. Both views agree that 101 coinbase outputs of 50 BTC exist and one is mature. The count is 101 and not 202 because each regtest coinbase also has a zero-value OP_RETURN witness-commitment output, which is provably unspendable and therefore never added to the UTXO set.

This also shows that a balance is a **derived** quantity: 50 BTC is the sum the wallet computes over its spendable UTXOs; no account balance is stored anywhere in the protocol.

2. Part B — a raw transaction built by hand

Step	Value
B1 input (<code>listunspent</code> 100)	txid 3b4aa4fcb007fa26cb484cc0164708cb39aaf3f0b98fb8643970e1649fc15b9f, vout 0, 50 BTC
B2 destination	bcrt1q455hejygdmf6pnsml2rc2qm98uqymuc9fejww
B2 change	bcrt1q20wj3prpnxsl4360x0zerc956dgpzcwh78kks

Amount computation (B3)

Input 50 BTC; payment 10 BTC; chosen fee 0.0001 BTC (10 000 sat). The fee is not written anywhere — it is whatever the outputs leave unclaimed — so the change output must absorb the rest:

change = 50 – 10 – 0.0001 = 39.9999 BTC → fee = $\Sigma \text{inputs} - \Sigma \text{outputs} = 50 - (10 + 39.9999) = \mathbf{0.0001 \text{ BTC}}$

`createrawtransaction` serialised one input and two outputs `{dest: 10.0}`, `{change: 39.9999}`; it performs no check of amounts against the input. `signrawtransactionwithwallet` returned `"complete": true` and a signed hex of **444** hex characters (222 bytes).

Result

Check	Observed	Expected
<code>decoderawtransaction</code>	outputs 10.00000000 and 39.99990000; size 222 , vsize 141 , weight 561 ; txid ≠ hash (SegWit)	222 / 141 / 561 ✓
<code>sendrawtransaction</code>	txid 398120acae03181b6cf8517f34f98ae9733af4bac8f6e24e5c ba02e1557ec911	—
<code>getrawmempool</code>	txid listed before mining; empty after <code>–generate</code> 1 (block 102)	✓
<code>gettransaction</code>	"fee": <code>–0.00010000</code> , "confirmations": 1, block height 102	–0.0001, 1 ✓
<code>getbalance</code>	99.99990000	≈ 99.9999 ✓

The node's fee equals my hand computation exactly: $|-0.00010000| = 0.0001 \text{ BTC} = 10\,000 \text{ sat}$. The sign is negative because the wallet reports the fee from my point of view, as value paid away. The effective fee rate

is **10 000 sat / 141 vB $\approx$ 70.9 sat/vB.**

The balance rose to 99.9999 BTC because both outputs (10 and 39.9999) went to my own wallet, and mining block 102 matured the block-2 coinbase (+50). The only value that left the wallet is the 0.0001 BTC fee, which — since I also mined block 102 — sits in a new immature coinbase and returns after 100 blocks.

3. Part C — my parser vs. decoderawtransaction

My parser (repository `lab-01-bitcoin-regtest`, standard library only) was run on the signed Part B hex. Full output:

```
=====
FIELD-BY-FIELD BREAKDOWN (offset = byte position in the serialised stream)
=====
offset  len  field                                     value
-----
    0    4  version                               2
    4    1  marker                               00
    5    1  flag                                0x01
    6    1  input count                           1
    7   32  input[0].prev_txid                    3b4aa4fcb007fa26cb484cc0164708cb39aaf3f0b98fb8643970e1649fc15b9f
   39    4  input[0].prev_vout                      0
   43    1  input[0].scriptSig length              0
   44    0  input[0].scriptSig
   44    4  input[0].sequence                        0xffffffff
   48    1  output count                           2
   49    8  output[0].value                       1000000000 sat
   57    1  output[0].scriptPubKey length          22
   58   22  output[0].scriptPubKey                 0014ad297cc8886ed33d0670dfd43c281b29f8026f98
   80    8  output[1].value                       3999990000 sat
   88    1  output[1].scriptPubKey length          22
   89   22  output[1].scriptPubKey                 001453dd28846199a1fac74f33c591e0b4d350608b0e
  111    1  witness[0] item count                  2
  112    1  witness[0][0] length                  71
  113   71  witness[0][0]                        3044022034325a3fbb4d9b0f889d053e61ef46b7ac1161878544b8f5c44c2e86...
(71 bytes)
  184    1  witness[0][1] length                   33
  185   33  witness[0][1]                        026ebd344d011c1de4835bb1c56fd295447e363c551fdb110bca2edab558f63f...
(33 bytes)
  218    4  locktime                             0

=====
SUMMARY
=====
txid      : 398120acae03181b6cf8517f34f98ae9733af4bac8f6e24e5cba02e1557ec911
wtxid     : 2bd2d821533dcb5f7937015b9b92d813fe901840b1420dca7007e8a7f118079b
version   : 2
segwit    : True (marker 0x00, flag 0x01 at offsets 4-5)
locktime  : 0 (none)
size      : 222 bytes
stripped size : 113 bytes
weight    : 561 WU (= 3 x 113 + 222)
vsize     : 141 vB

INPUTS (1)
[0] at offset 7
  prev txid : 3b4aa4fcb007fa26cb484cc0164708cb39aaf3f0b98fb8643970e1649fc15b9f
  prev vout : 0
  scriptSig : (empty)
  sequence  : 0xffffffff (RBF signalled, BIP-125)
  witness   : 2 item(s)
    [0] 71 bytes DER signature, sighash 0x01 (SIGHASH_ALL)
        3044022034325a3fbb4d9b0f889d053e61ef46b7ac1161878544b8f5c44c2e86b8f345ce022068fff26ec5368e... (71 bytes)
    [1] 33 bytes compressed public key
        026ebd344d011c1de4835bb1c56fd295447e363c551fdb110bca2edab558f63f0f

OUTPUTS (2)
[0] at offset 49
  value      : 10.00000000 BTC (1000000000 sat)
  scriptPubKey : 0014ad297cc8886ed33d0670dfd43c281b29f8026f98
  asm        : OP_0 <ad297cc8886ed33d0670dfd43c281b29f8026f98>
  type       : p2wpkh
  address    : bcrt1q455hejygdmf6pnsml2rc2qm98uqymuc9fejjw
[1] at offset 80
  value      : 39.99990000 BTC (3999990000 sat)
  scriptPubKey : 001453dd28846199a1fac74f33c591e0b4d350608b0e
  asm        : OP_0 <53dd28846199a1fac74f33c591e0b4d350608b0e>
```

```
type      : p2wpkh
address   : bcrt1q20wj3prpnxsl4360x0zerc956dgpzcwh78kks
total output : 4999990000 sat
```

FEE

not computable from this transaction alone. Bitcoin has no fee field:
fee = sum(input values) - sum(output values), and input values are not
serialised here -- each lives in the output being spent (prev txid:vout).
Looking those up requires a node or UTXO set.

Side-by-side comparison

Field	decoderawtransaction (node)	My parser	
txid	398120ac...557ec911	398120ac...557ec911	=
wtxid	hash: 2bd2d821...f118079b	wtxid: 2bd2d821...f118079b	=
version / locktime	2 / 0	2 / 0	=
size / vsize / weight	222 / 141 / 561	222 / 141 / 561	=
stripped size	not shown	113 (561 = 3 × 113 + 222)	+
SegWit marker/flag	implicit	0x00 / 0x01 at offsets 4–5	+
input[0] prev txid	3b4aa4fc...9fc15b9f, vout 0	same, at offset 7	=
sequence	4294967293 (decimal)	0xffffffff (hex, RBF signalled)	≈
witness	2 items, hex only	71-byte DER sig ending 01 (SIGHASH_ALL), 33-byte key 02...	=
output scripts	0 ad297cc8..., type witness_v0_keyhash	OP_0 <ad297cc8..., type p2wpkh	≈
output values	10.00000000 / 39.99990000 BTC	1 000 000 000 / 3 999 990 000 sat	≈
addresses	bcrt1q455hej..., bcrt1q20wj3p...	identical (own Bech32 encoder)	=
byte offsets	not shown	every field	+
fee	not shown	explicitly not computable	=

= identical · ≈ same value, different representation · + only my parser shows it

Where the two representations differ

The hashes. The node labels the witness hash hash; my parser calls it wtxid. Both are double-SHA-256 values, but over different bytes: the txid is computed over the legacy serialisation *without* marker, flag and witness (113 bytes), the wtxid over the full 222 bytes. Because this is a SegWit transaction, they differ; for a legacy transaction they would be equal (tested on the genesis and block-170 transactions).

Byte order. Every hash is stored little-endian but displayed big-endian. The previous txid appears in the raw bytes at offset 7 as 9f5bc19f64e17039...b0fca44a3b, which is the byte-reversed form of the displayed 3b4aa4fc b007fa26...649fc15b9f. The same reversal is applied to the computed txid and wtxid before printing; without it, neither would match the node.

Notation. The node prints the sequence in decimal (4294967293), amounts in BTC, OP_0 as 0 and uses Core's type names; my parser prints hex, satoshis, full opcode mnemonics and the lab's short type names. The underlying values are identical.

Result: my parser's txid equals the one returned by sendrawtransaction, and its wtxid equals the node's hash — for Part B and also for the Part D transaction (1e083cb6...3f5a3ba3 / c75065b9...82917f2a).

Test suites

My implementation passes the course's reference test suite of 31 tests (genesis coinbase, block-170 payment, SegWit fixture, CompactSize, scripts, error handling) plus 60 own tests: p2sh and p2wsh classification with BIP-173 vectors, all CompactSize boundaries (0, 252, 253, 65 535, 65 536, $2^{32}-1$, 2^{32}) at non-zero offsets, rejection of non-canonical and truncated encodings, first-input offsets 5 (legacy) and 7 (SegWit), and robustness against truncated input, trailing bytes and non-hex input — **91 passed**. The reference parser's own suite also ran: **31 passed**.

4. Part D — the deliberate mistake

I repeated Part B with the now-mature block-2 coinbase (66477223...b4b86ea5:0, 50 BTC) but with a single output of 10 BTC to bcrt1qmm4drjz0...flhjwq and **no change output**.

Step	Observed
1st sendrawtransaction	error code: -25 — Fee exceeds maximum configured by user (e.g. -maxtx fee, maxfeerate)
2nd, maxfeerate 0	accepted, txid 1e083cb63b9c64838d379b047a87c53420c13a7e9a76e9fadc1b27893f5a3ba3
-generate 1	block 103, hash 55a067a9...b8c1030b, 2 transactions
block tx[0] (coinbase)	vout[0] = 90.00000000 → bcrt1q88el447z...c23l87r (my mining address); vout[1] = 0.00000000, OP_RETURN witness commitment
block tx[1] (mine)	one output of 10.00000000 BTC; block reports "fee": 40.00000000; vsize 110
gettransaction	"fee": -40.00000000

The first refusal is **not** a consensus rule. It is a safety limit in Bitcoin Core's RPC (maxfeerate, default 0.10 BTC/kvB). This transaction pays 40 BTC for 110 vB — about 36 million sat/vB, roughly 500 000 times the 71 sat/vB of Part B — so the RPC refuses it. Passing 0 disables the check, and the network accepted the transaction.

Who received the 40 BTC

The miner of block 103 — whoever was paid by that block's coinbase. The coinbase claims 90 BTC = 50 BTC subsidy + 40 BTC fee, paid to the address given to -generate, which is in my own wallet. The coins therefore return to me as an immature coinbase: after block 103, getbalances shows immature **5040.0001** BTC ($100 \times 50 + 40 + 0.0001$, the last two being the Part D and Part B fees) and trusted **109.9999** BTC (3 mature coinbases $\times 50 - 40 - 0.0001$). On mainnet the miner is an unrelated party and the 40 BTC would be lost.

Why the transaction was consensus-valid

Consensus checks value with two inequalities: for each transaction $\Sigma \text{inputs} \geq \Sigma \text{outputs}$ ($50 \geq 10$), and for each block the coinbase may claim no more than subsidy + Σfees ($90 \leq 50 + 40$). Both hold. Nothing in consensus limits how large the difference may be, because the protocol cannot know whether a large remainder is a mistake or an intentional payment to the miner. The signature committed to exactly these outputs, so the transaction is valid as signed.

Why Ethereum's account model makes this error impossible

Ethereum's state is a mapping from accounts to balances, and a transaction is a state transition rather than the consumption of whole outputs. For a simple transfer the transition function does $\text{balance}[\text{sender}] -= \text{value} + \text{gas_used} \times \text{gas_price}$ and $\text{balance}[\text{recipient}] += \text{value}$. The sender states only the amount to

move; everything not moved simply *stays* in the sender's account. There is no set of outputs that must be balanced against the inputs, so there is no remainder that can be forgotten. The fee is not implicit either: it is derived from explicit, signed fields and bounded in advance by $\text{gasLimit} \times \text{maxFeePerGas}$ (unused gas is refunded), so the sender can never pay more than that cap. In Bitcoin's UTXO model an input is consumed whole and anything not re-assigned to an output becomes the fee; in Ethereum, anything not transferred remains the sender's balance.

5. Why Bitcoin has no explicit fee field

Bitcoin transactions carry no fee field because the fee is already fully determined by data the network holds. Every input points to an earlier output, and that output's value is fixed in the UTXO set; the transaction cannot change it. Every validating node therefore knows the sum of the inputs and reads the sum of the outputs, so $\text{fee} = \Sigma \text{inputs} - \Sigma \text{outputs}$ follows without any extra information. An explicit field would be redundant, and worse, it could contradict the implicit value. The protocol would then need one more consensus rule to say which number wins.

The design buys simplicity in several places. Validation of value is a single inequality per transaction: $\Sigma \text{inputs} \geq \Sigma \text{outputs}$. The miner's reward needs no fee bookkeeping either: the coinbase may claim the block subsidy plus the sum of all remainders, which Part D showed as $50 + 40 = 90$ BTC. Pricing is left entirely to the sender, who sets the fee rate (sat/vB) through the size of the remainder; a fee market emerges without the protocol fixing any price. Finally, every transaction is a few bytes smaller.

The cost is that the fee is invisible and easy to get wrong. A sender who forgets the change output does not get an error from consensus; the transaction is valid, and the whole remainder goes to the miner, exactly as in Part D, where 40 BTC was lost to a fee. The fee also cannot be read from the transaction alone: our Part C parser has to refuse to print it, because the input values live in other transactions. This is why wallets and Bitcoin Core add guard rails outside consensus, such as the `maxfeerate` check that rejected Part D with error -25.

(288 words)